Supervised Learning Setup

Dataset $\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$; goal: learn f s.t. $\hat{y} = f(\mathbf{x}) \approx y$ on *unseen* data.

Regression: $y \in \mathbb{R}$ (continuous).
Classification: $y \in \{1, \dots, c\}$. CIFAR-10: 60k imgs, 32×32×3 (reshape → 3072-dim vector), 10 classes, 6k/class; 50k train / 10k test.

Linear / Least Squares Regression

Model: $\hat{y} = ax + b = \theta^T \hat{\mathbf{x}}$, $\theta = [a, b]^T$, $\hat{\mathbf{x}} = [x, 1]^T$. Polynomial: $\theta = [b, a_1, \dots, a_n]^T$, $\hat{\mathbf{x}} = [1, x, x^2, \dots, x^n]^T$ (same notation, still linear in θ). Loss (N = normalization const):

$$\mathcal{L}(\theta) = \frac{1}{2N} \sum_{i=1}^{N} \left(y^{(i)} - \theta^T \hat{\mathbf{x}}^{(i)}\right)^2 = \frac{1}{2N} (\mathbf{y} - \mathbf{X}\theta)^T (\mathbf{y} - \mathbf{X}\theta)$$

$\mathbf{X} \in \mathbb{R}^{N \times n}$ rows $\hat{\mathbf{x}}^{(i)T}$. Expand: $\frac{1}{2N}(\mathbf{y}^T\mathbf{y} - 2\mathbf{y}^T\mathbf{X}\theta + \theta^T\mathbf{X}^T\mathbf{X}\theta)$.

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{1}{2N}(-2\mathbf{X}^T\mathbf{y} + 2\mathbf{X}^T\mathbf{X}\theta) = 0$$

$$\boxed{\mathbf{X}^T\mathbf{X}\theta = \mathbf{X}^T\mathbf{y} \;\Rightarrow\; \theta = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}}$$

Higher-degree poly always fits *train* data at least as well (zero extra coeffs), but may not generalize.

Vector/Matrix Derivatives

Denominator layout (used in class): $\nabla_{\mathbf{x}} y$ has *same shape as* $\mathbf{x}$. $y \in \mathbb{R}$, $\mathbf{x} \in \mathbb{R}^n$: $\nabla_{\mathbf{x}} y = \left[\frac{\partial y}{\partial x_1}, \dots, \frac{\partial y}{\partial x_n}\right]^T$.
$\Delta y \approx \sum_i \frac{\partial y}{\partial x_i} \Delta x_i = (\nabla_{\mathbf{x}} y)^T \Delta\mathbf{x}$.

- $\nabla_{\mathbf{x}}(\theta^T\mathbf{x}) = \theta, \quad \nabla_\theta(\theta^T\mathbf{x}) = \mathbf{x}$
- $\nabla_{\mathbf{x}}(\mathbf{x}^T\mathbf{A}\mathbf{x}) = (\mathbf{A} + \mathbf{A}^T)\mathbf{x}$; $= 2\mathbf{A}\mathbf{x}$ if $\mathbf{A} = \mathbf{A}^T$
- $\nabla_{\mathbf{x}}(\mathbf{W}\mathbf{x}) = \mathbf{W}^T$ ($\mathbf{W} \in \mathbb{R}^{h \times n}$)
- $\nabla_{\mathbf{A}} y \in \mathbb{R}^{m \times n}$ for $\mathbf{A} \in \mathbb{R}^{m \times n}$, (i, j) entry $\partial y / \partial a_{ij}$. E.g. $y = \mathbf{z}^T\mathbf{A}\mathbf{x} \Rightarrow \nabla_{\mathbf{A}} y = \mathbf{z}\mathbf{x}^T$
- Facts: $(\mathbf{ABC})^T = \mathbf{C}^T\mathbf{B}^T\mathbf{A}^T$; $s^T = s$ for scalar

Jacobian: $\mathbf{y} \in \mathbb{R}^n, \mathbf{x} \in \mathbb{R}^m$: $\Delta\mathbf{y} \approx \mathbf{J}\Delta\mathbf{x}$, $\mathbf{J} \in \mathbb{R}^{n \times m}$, $J_{ij} = \frac{\partial y_i}{\partial x_j}$. Denominator layout: $\nabla_{\mathbf{x}}\mathbf{y} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \in \mathbb{R}^{m \times n} = \mathbf{J}^T$.

Hessian: $\mathbf{H} = \nabla_{\mathbf{x}}(\nabla_{\mathbf{x}} f) = \nabla_{\mathbf{x}}^2 f$, $H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$.

Generalization

Overfit: low train err, high test err (too complex). **Underfit**: high train err (too simple, e.g. linear for nonlinear data). More data ⇒ can fit more complex models w/o overfitting. Assumption: test data from *same distribution* as train; else "out-of-distribution." Over/underfit ↔ *bias/variance* of param. estimate.

Train (learn params) / **Validation** (tune hyperparams) / **Test** (score once; untouched during model discovery).

k-fold CV: split train data into k folds; train on $k-1$, validate on 1; rotate, average val. err. → pick hyperparams → retrain on all folds → evaluate once on test. E.g. CIFAR: 10k test; 50k→5 folds of 10k (40k train/10k val). For NNs often just one val set (cost). Tiny data: model selection (BIC, AIC, DIC) penalizes complexity.

Image Classification & k-NN

Challenges: viewpoint, zoom, illumination, deformation, occlusion, background clutter, intraclass variation. Data-driven: learn features.

k-NN: find the k closest training pts (under a metric); majority vote. "Training" = store data. Hyperparams: distance metric, k. Larger k smooths outliers. Pros: simple, no training. Cons: memory (keep all data), slow inference, curse of dim. (volume grows exponentially → neighbors not near); hard to pick metric (boxed/shifted/tinted imgs: equal L_2 dist.). Cover–Hart: $\text{ERR}_{\text{NN}} \le 2\,\text{ERR}_{\text{opt}} + \frac{2c\sqrt{d}}{N^{1/d}}$ (slow decay in N).

Linear Classifier

$\mathbf{W} = [\mathbf{w}_1^T; \dots; \mathbf{w}_c^T] \in \mathbb{R}^{c \times n}$,
$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b} \in \mathbb{R}^c$ (scores), predict $k = \arg\max_j z_j$. Each row $\mathbf{w}_i$ = class *template*; $\mathbf{w}_i^T\mathbf{x} = \|\mathbf{w}_i\|\|\mathbf{x}\| \cos\theta_i$ large when $\mathbf{w}_i$ nearly colinear with $\mathbf{x}$. Boundary (2 classes, no bias): $(\mathbf{w}_1 - \mathbf{w}_2)^T\mathbf{x} = 0$; class 1 if $(\mathbf{w}_1 - \mathbf{w}_2)^T\mathbf{x} \ge 0$. Fails when data not linearly separable (e.g. concentric rings) → change of variables (polar coords) / NN learns the representation.

Maximum Likelihood

Choose θ maximizing $P(\mathcal{D}|\theta)$. Coin: $\mathcal{L}(\theta) = \theta^{n_H}(1-\theta)^{n_T}$;
$\log\mathcal{L} = n_H \log\theta + n_T \log(1-\theta)$;
$\frac{\partial}{\partial\theta} = 0 \Rightarrow \theta^* = \frac{n_H}{n_H + n_T}$.
$\arg\max\mathcal{L} = \arg\max\log\mathcal{L}$ (log monotone).
Chain rule: $p(a,b) = p(a)p(b|a)$;
$p(a,b,c) = p(a)p(b|a)p(c|a,b)$;
$p(b,c|d,e) = \frac{p(b,c,d,e)}{p(d,e)}$. i.i.d.:
$P(\mathcal{D}|\theta) = \prod_i p(\mathbf{x}^{(i)}|\theta)\, p(y^{(i)}|\mathbf{x}^{(i)}, \theta)$; $p(\mathbf{x}|\theta)$ indep. of $\theta \Rightarrow \arg\max_\theta \sum_i \log p(y^{(i)}|\mathbf{x}^{(i)}, \theta)$.

Softmax Classifier

$a_i(\mathbf{x}) = \mathbf{w}_i^T\mathbf{x} + b_i$, $\theta = \{\mathbf{w}_j, b_j\}_{j=1}^c$:

$$\text{softmax}_i(\mathbf{x}) = \frac{e^{a_i(\mathbf{x})}}{\sum_{j=1}^c e^{a_j(\mathbf{x})}} \in [0,1], \;\; \sum_i = 1$$

$\Pr(y^{(i)} = j|\mathbf{x}^{(i)}, \theta) = \text{softmax}_j(\mathbf{x}^{(i)})$.
Loss (neg. log-likelihood = cross-entropy):

$$J(\theta) = \frac{1}{m}\sum_{i=1}^m \left(\log\sum_{j=1}^c e^{a_j(\mathbf{x}^{(i)})} - a_{y^{(i)}}(\mathbf{x}^{(i)})\right) = -\sum_i \sum_j z_j^{(i)} \log p_j^{(i)}$$

where $p_j^{(i)} = \Pr(y^{(i)}{=}j|\mathbf{x}^{(i)}, \theta)$. $\mathbf{z}^{(i)}$ one-hot: $z_j^{(i)} = 1$ iff $j = y^{(i)}$ (only correct-class term survives). $\log\Pr(y^{(i)}|\mathbf{x}) = a_{y^{(i)}} - \log\sum_j e^{a_j} \approx a_{y^{(i)}} - \max_j a_j$: ≈0 if correct class wins, else < 0.

Overflow: e^{a_i} huge. Multiply num/denom by k: $\frac{e^{a_i + \log k}}{\sum_j e^{a_j + \log k}}$, $\log k = -\max_j a_j(\mathbf{x})$ (max exponent = 0).

Gradient Descent

Taylor: $f(\mathbf{x} + \Delta\mathbf{x}) \approx f(\mathbf{x}) + \Delta\mathbf{x}^T\nabla_{\mathbf{x}} f(\mathbf{x})$. Directional deriv. along unit $\mathbf{u}$: $\mathbf{u}^T\nabla_{\mathbf{x}} f$. $\min_{\|\mathbf{u}\|=1} \mathbf{u}^T\nabla f = \min\|\nabla f\|\cos\alpha \Rightarrow \mathbf{u} = -\nabla f$ ($\cos\alpha = -1$).

$$\boxed{\mathbf{x} \leftarrow \mathbf{x} - \epsilon\nabla_{\mathbf{x}} f(\mathbf{x})} \quad \epsilon = \text{learning rate}$$

Terms: global min $f(\mathbf{x}) \ge f(\mathbf{x}_g)\,\forall\mathbf{x}$; local min = critical pt lower than neighbors; saddle = critical pt, neither max nor min ($f' = 0$ at all). NN losses non-convex; local minima empirically good.

Step size: too large → Taylor approx invalid, diverge/bounce; too small → slow. LR schedule/annealing: start large, decay when loss plateaus. Line search too costly in high dim.

Numerical grad $\frac{f(x+h) - f(x)}{h}$: OK for small models (grad check), too expensive for large.

Batch: all m examples. **Minibatch**: k examples ($m > k > 1$, e.g. 32/64/128).

Stochastic: $k = 1$.
$\nabla_\theta\mathcal{L} = \frac{1}{N}\sum_i \nabla_\theta \ell(\mathbf{x}^{(i)}, y^{(i)}) \approx \mathbb{E}_{\mathbf{x},y}[\nabla_\theta\ell]$; each sample = noisy grad estimate. SGD noise can kick out of bad local minima; implicit regularization.

Neural Networks

Neuron: $f(\sum_i w_i x_i + b)$ (synaptic weights → hillock integrate+threshold → axon). Real neurons spike (all-or-nothing, probabilistic); NNs encode the *rate*.

MLP / feedforward: $\mathbf{h}_1 = f(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$, $\mathbf{h}_2 = f(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2)$, …, $\mathbf{z} = \mathbf{W}_N\mathbf{h}_{N-1} + \mathbf{b}_N$ (no activation on output; scores go to softmax/SVM). f applied elementwise.

Param count: $3 \to 4 \to 2$: $3{\cdot}4 + 4{\cdot}2 = 20$ W, $4{+}2 = 6$ b = 26. $3 \to 4 \to 4 \to 2$: $12 + 16 + 8 = 36$ W $+10$ b = 46. (#neurons excludes input.) Universal approx.: 2-layer MLP w/ enough neurons approximates any function.

Linear f: collapses to $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$, $\mathbf{W} = \mathbf{W}_N \cdots \mathbf{W}_1$, $\mathbf{b} = \mathbf{b}_N + \mathbf{W}_N\mathbf{b}_{N-1} + \cdots + \mathbf{W}_N \cdots \mathbf{W}_2\mathbf{b}_1$. Useful only for low-rank rep. (autoencoder, $\dim\mathbf{h} \ll \dim\mathbf{x}$, $\mathcal{L} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$, ∼SVD). Nonlinear hidden layers = learned features making data linearly separable for last layer.

XOR: linear LS fit fails ($w_1{=}w_2{=}w$, $b = \frac{1-3w}{2}$). Works with $g(\mathbf{x}) = \mathbf{w}^T \max(0, \mathbf{W}^T\mathbf{x} + \mathbf{c}) + b$, $\mathbf{W} = \begin{bmatrix}1 & 1\\ 1 & 1\end{bmatrix}$, $\mathbf{c} = [0, -1]^T$, $\mathbf{w} = [1, -2]^T$, $b = 0$.

Activation Functions

Principle: gradient must be "large and predictable enough" to guide learning.

Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$, $\sigma' = \sigma(1-\sigma)$
tanh	$2\sigma(2x) - 1$, $\tanh' = 1 - \tanh^2$
ReLU	$\max(0, x)$, $' = \mathbb{I}(x > 0)$
Softplus	$\zeta(x) = \log(1 + e^x)$, $\zeta' = \sigma(x)$
Leaky/PReLU	$\max(\alpha x, x)$, $\alpha \ll 1$ (0.01)
ELU	$\max(\alpha(e^x - 1), x)$
Swish	$x\,\sigma(\beta x)$
GELU	$x\Phi(x) = \frac{x}{2}[1 + \text{erf}(x/\sqrt{2})]$

Sigmoid: +linear near 0, differentiable. −saturates (zero grad at extremes); not zero-centered (outputs ∼0.5): for $f = \sigma(\mathbf{w}^T\mathbf{h} + b)$, $\frac{\partial f}{\partial \mathbf{w}} = \sigma(1-\sigma)\mathbf{h}$; if $\mathbf{h} \ge 0$ all grad entries share sign → zig-zag. Rarely used.

tanh: zero-centered (fixes zig-zag) but still saturates.

ReLU: converges faster (AlexNet ∼6× vs tanh), no saturation for $x > 0$, large grads. −not zero-centered, not diff. at 0 (use 0 or 1; subgradient ∈ [0, 1]), no learning when activation = 0 (dead units) → leaky ReLU/maxout. Softplus: smooth but empirically worse than ReLU. PReLU: α learned. ELU: needs costly exp.

Practice: ReLU popular; sigmoid almost never (tanh preferred); try leaky ReLU/PReLU/ELU/maxout.

Output Activation ↔ Cost

Binary, $\hat{y} = \sigma(z)$, $y \in \{0, 1\}$:
MSE $= \frac{1}{2}\sum_i (y^{(i)} - \sigma(z^{(i)}))^2$;
CE $= -\sum_i [y \log \sigma(z) + (1-y)\log(1-\sigma(z))]$.
$y = 1$:
$\frac{\partial \text{MSE}}{\partial z} = -(y - \sigma(z))\,\sigma(z)(1-\sigma(z)) \to 0$ when $z \ll 0$ (very wrong yet *no learning*).
$\frac{\partial \text{CE}}{\partial z} = \sigma(z) - 1 \approx -1$ when $z \ll 0$ (learns); stalls only near right answer. ⇒ CE well matched to σ. Choose loss & nonlinearity jointly by looking at the gradient.

Backpropagation

Repeated chain rule on a computational graph; cache forward values.
Forward: compute node values.
Backward: downstream grad = local grad × upstream grad: $\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial f}{\partial x}\frac{\partial \mathcal{L}}{\partial f}$.
Gates: **add** distributes ($\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} = \frac{\partial \mathcal{L}}{\partial f}$); **mult** switches ($\frac{\partial \mathcal{L}}{\partial x} = y\frac{\partial \mathcal{L}}{\partial f}$, $\frac{\partial \mathcal{L}}{\partial y} = x\frac{\partial \mathcal{L}}{\partial f}$); **max** routes ($\mathbb{I}(x > y)\frac{\partial \mathcal{L}}{\partial f}$, $\mathbb{I}(y \geq x)\frac{\partial \mathcal{L}}{\partial f}$).
Paths converge: sum. $\frac{\partial \mathcal{L}}{\partial x} = \sum_i \frac{\partial q_i}{\partial x}\frac{\partial \mathcal{L}}{\partial q_i}$.
Ex. $f = (x + y)z$, x=2, y=3, z=4: w=5, f=20; $\frac{\partial f}{\partial z} = w = 5$, $\frac{\partial f}{\partial w} = z = 4$, $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial y} = 4$.
Ex. $f = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$, w_0=1, x_0=−1, w_1=2, x_1=2, w_2=−2: fwd $-1, 4 \to 3 \to 1 \to -1 \to 0.37 \to 1.37 \to 0.73$. Back: inv $(-1/z^2)$ −0.53; +1: −0.53; exp: −0.20; ×(−1): 0.20; adds pass 0.20. $\frac{\partial f}{\partial w_0} = -0.20$, $\frac{\partial f}{\partial x_0} = 0.20$, $\frac{\partial f}{\partial w_1} = 0.40$, $\frac{\partial f}{\partial x_1} = 0.40$, $\frac{\partial f}{\partial w_2} = 0.20$. (Shortcut: $\sigma' = \sigma(1-\sigma) = 0.73 \cdot 0.27 \approx 0.20$.)
Softplus: $b = e^x$, $a = 1 + b$, $y = \log a$: $\frac{dy}{da} = \frac{1}{a}$, $\frac{dy}{dx} = e^x\frac{1}{a}$.

Vector chain rule

Scalar: $\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$. Vector: $\mathbf{x} \in \mathbb{R}^m, \mathbf{y} = f(\mathbf{x}) \in \mathbb{R}^n, \mathbf{z} = g(\mathbf{y}) \in \mathbb{R}^p$:

$$\nabla_{\mathbf{x}}\mathbf{z} = \nabla_{\mathbf{x}}\mathbf{y}\,\nabla_{\mathbf{y}}\mathbf{z}, \quad \frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}}\frac{\partial \mathbf{z}}{\partial \mathbf{y}}$$

$(m\times p) = (m\times n)(n\times p)$. **Order matters**: right-to-left in denominator layout; check dims. (From $\Delta\mathbf{z} \approx (\nabla_{\mathbf{y}}\mathbf{z})^T(\nabla_{\mathbf{x}}\mathbf{y})^T\Delta\mathbf{x}$.)
Tensor derivs ($\partial\mathbf{y}/\partial\mathbf{W}$: vector w.r.t. matrix is a 3-D tensor; not tested). Avoid by differentiating the scalar loss directly:

$$\mathbf{y} = \mathbf{W}\mathbf{x}: \quad \frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{W}^T\frac{\partial \mathcal{L}}{\partial \mathbf{y}}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}}\mathbf{x}^T$$

NN layer backprop

$\mathbf{h} = \text{ReLU}(\mathbf{W}\mathbf{x} + \mathbf{b})$, $\ell = \|\mathbf{h} - \mathbf{z}\|^2$; $\mathbf{c} = \mathbf{W}\mathbf{x}$, $\mathbf{a} = \mathbf{c} + \mathbf{b}$. $\nabla_{\mathbf{h}}\ell = 2(\mathbf{h} - \mathbf{z})$.

$$\frac{\partial \ell}{\partial \mathbf{a}} = \mathbb{I}(\mathbf{a} > 0) \odot \frac{\partial \ell}{\partial \mathbf{h}}, \quad \frac{\partial \ell}{\partial \mathbf{c}} = \frac{\partial \ell}{\partial \mathbf{b}} = \frac{\partial \ell}{\partial \mathbf{a}}$$

$$\frac{\partial \ell}{\partial \mathbf{x}} = \mathbf{W}^T\frac{\partial \ell}{\partial \mathbf{c}}, \quad \frac{\partial \ell}{\partial \mathbf{W}} = \frac{\partial \ell}{\partial \mathbf{c}}\mathbf{x}^T$$

$\odot$ = Hadamard (elementwise). Naive GD on NNs performs poorly → need init, normalization, regularization, better optimizers.

Weight Initialization

Small (e.g. $0.01 \cdot \mathcal{N}(0, 1)$): activations decay to 0 through layers ⇒ $\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}}\mathbf{h}^T \approx 0$, no learning. OK for shallow nets.
Large ($\mathcal{N}(0, 1)$): activations explode (ReLU) / saturate at ±1 (tanh) ⇒ noisy/vanishing grads. **Xavier** (Glorot & Bengio): keep $\text{var}(h_i) = \text{var}(h_j)$ and $\text{var}(\nabla_{h_i} J) = \text{var}(\nabla_{h_j} J)$ across layers. Linear units $h_i = \sum_{j=1}^{n_{\text{in}}} w_{ij}h_{i-1,j}$; w, h indep., zero mean:

$$\text{var}(wh) = \mathbb{E}^2(w)\text{var}(h) + \mathbb{E}^2(h)\text{var}(w) + \text{var}(w)\text{var}(h)$$

$\Rightarrow \text{var}(h_i) = \text{var}(h_{i-1})\sum_j \text{var}(w_{ij}) \Rightarrow \text{var}(w) = \frac{1}{n_{\text{in}}}$. Backward: $\frac{1}{n_{\text{out}}}$. Compromise:

$$\text{var}(w_{ij}) = \frac{2}{n_{\text{in}} + n_{\text{out}}}, \quad w \sim \mathcal{N}(0, \frac{2}{n_{\text{in}} + n_{\text{out}}})$$

or $U\left(\pm\sqrt{6/(n_{\text{in}} + n_{\text{out}})}\right)$. Good for tanh; with ReLU → dying units.
He/MSRA (ReLU): $\text{var}(w) = \frac{2}{n_{\text{in}}}$ (ReLU kills half the units, halving variance). Code: `randn(..)*sqrt(2)/sqrt(n_in)`. A factor of 2 can decide learning vs. not.

Batch Normalization

(Ioffe & Szegedy 2015) Motivated by internal covariate shift (newer view: smoother loss surface). Per unit i, minibatch size m:

$$\mu_i = \frac{1}{m}\sum_j x_i^{(j)}, \quad \sigma_i^2 = \frac{1}{m}\sum_j (x_i^{(j)} - \mu_i)^2$$

$$\hat{x}_i = \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}, \qquad y_i = \gamma_i\hat{x}_i + \beta_i$$

γ, β learned; can undo BN ($\gamma_i = \sigma_i, \beta_i = \mu_i$). Per-unit (not full $\Sigma^{-1/2}$) for computational efficiency. Part of graph → backprop through it.
Backprop (drop i):

$$\frac{\partial \ell}{\partial \beta} = \sum_j \frac{\partial \ell}{\partial y^{(j)}}, \quad \frac{\partial \ell}{\partial \gamma} = \sum_j \frac{\partial \ell}{\partial y^{(j)}}\hat{x}^{(j)}$$

$$\frac{\partial \ell}{\partial \hat{x}^{(j)}} = \gamma\frac{\partial \ell}{\partial y^{(j)}}$$

$$\frac{\partial \ell}{\partial \sigma^2} = \sum_j -\frac{1}{2}\frac{(x^{(j)} - \mu)}{(\sigma^2 + \varepsilon)^{3/2}}\frac{\partial \ell}{\partial \hat{x}^{(j)}}$$

$$\frac{\partial \ell}{\partial \mu} = -\frac{1}{\sqrt{\sigma^2 + \varepsilon}}\sum_j \frac{\partial \ell}{\partial \hat{x}^{(j)}} - \frac{2}{m}\sum_j (x^{(j)} - \mu)\frac{\partial \ell}{\partial \sigma^2}$$

$$\frac{\partial \ell}{\partial x^{(j)}} = \frac{1}{\sqrt{\sigma^2 + \varepsilon}}\frac{\partial \ell}{\partial \hat{x}^{(j)}} + \frac{2(x^{(j)} - \mu)}{m}\frac{\partial \ell}{\partial \sigma^2} + \frac{1}{m}\frac{\partial \ell}{\partial \mu}$$

Placement: $\mathbf{h}_i = f(\text{BN}(\mathbf{W}_i\mathbf{h}_{i-1} + \mathbf{b}_i))$ (before nonlinearity). **Test time**: use running avg of mean/var over seen inputs (forgetting factor $\mu_t = \sum_j \alpha^{t-j}x^{(j)}$). Allows higher LRs.

Regularization

"Any modification ... intended to reduce generalization error but not training error." Increases bias, reduces variance; prevents overfitting; lets you use a big model regularized.
Early stopping: return model w/ lowest val. error; #steps is a hyperparam; no change to model/cost.
Norm penalty: $\tilde{J} = J(\theta; \mathbf{X}, \mathbf{y}) + \alpha\Omega(\theta)$, $\alpha \geq 0$ (0: none; ∞: just min Ω).
L^2 (ridge/Tikhonov/weight decay): $\Omega = \frac{1}{2}\mathbf{w}^T\mathbf{w}$. $\nabla_{\mathbf{w}}\tilde{J} = \alpha\mathbf{w} + \nabla_{\mathbf{w}}J$; $\mathbf{w} \leftarrow (1 - \epsilon\alpha)\mathbf{w} - \epsilon\nabla_{\mathbf{w}}J$ (shrink, then step). (Not tested: MAP w/ $\mathbf{w} \sim \mathcal{N}(0, \frac{1}{\alpha}\mathbf{I})$; LS becomes $\mathbf{w} = (\mathbf{X}^T\mathbf{X} + \alpha\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$.) Variants: $\|\mathbf{w} - \mathbf{b}\|_2$ (near prior value), $\|\mathbf{w}^{(1)} - \mathbf{w}^{(2)}\|_2$ (tie params).
L^1: $\Omega = \|\mathbf{w}\|_1 = \sum_i |w_i|$; subgrad $\text{sign}(\mathbf{w})$ (size-independent) ⇒ *sparse* $\mathbf{w}$ (feature selection). MAP w/ $w_i \sim \text{Laplace}(0, \frac{1}{\alpha})$.
Sparse reps: $\Omega(\mathbf{h}^{(i)}) = \|\mathbf{h}^{(i)}\|_1$.
Data aug. (encodes invariances): flip, crop, brightness, lens corr., rotate; noise injection, label smoothing.
Multitask: shared layers + task-specific heads (shared factors). **Transfer**: reuse layers from another task (fixed), train a few task layers.
Ensembles: train several models, average at test (few % gain). **Bagging**: k datasets resampled w/ replacement. Cheap: snapshots at different local minima.

Dropout

Each iter., drop each unit independently: `M=(rand(*H.shape)<p); H*=M`.
Test: weight scaling inference rule — $h_{\text{out}} = \text{relu}(p\sum_i w_ih_i) = p\,\text{relu}(\sum w_ih_i)$; class scales activations by p.
Inverted dropout: scale by $1/p$ in training (`M=(rand<p)/p`) ⇒ same expected value; test unchanged. No extra test cost (vs. $O(m)$ for m-model ensemble). Why: ≈bagging (2^N sub-models, N units); units work in many contexts; redundant features.

SGD & Momentum

SGD: LR ε, init θ, minibatch m; until stop: sample $\{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^m$,

$$\boxed{\mathbf{g} = \frac{1}{m}\sum_{i=1}^m \nabla_\theta J(\theta), \quad \theta \leftarrow \theta - \varepsilon\mathbf{g}}$$

Momentum: running mean of grads. $\mathbf{v} = 0$, $\alpha \in [0, 1]$ (0.9/0.99):

$$\boxed{\mathbf{v} \leftarrow \alpha\mathbf{v} - \varepsilon\mathbf{g}, \quad \theta \leftarrow \theta + \mathbf{v}}$$

$\mathbf{v}_1 = -\varepsilon\mathbf{g}_1$, $\mathbf{v}_3 = -\varepsilon[\alpha^2\mathbf{g}_1 + \alpha\mathbf{g}_2 + \mathbf{g}_3]$. Averages away zig-zag on high-curvature tilted surfaces; can push to a different (local) minimum.